



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
INGENIERÍA INFORMÁTICA
LENGUAJES Y COMPILADORES

DIFERENCIAS EN LA TOKENIZACIÓN ENTRE NLP Y LENGUAJES FORMALES Y SU IMPLICACIÓN CON LOS COMPILADORES

DOCENTE:
FELIX MARQUEZ

GRUPO NABLA:

DANIEL VILLALBA SANTAMARIA
ISABELLA RODRIGUEZ
ANDRES GOMEZ
LUIS MILLAN

CIUDAD GUAYANA, MAYO DEL 2026



LENGUAJES Y COMPILADORES

TOKENIZACION

Consiste en el proceso de transformar una secuencia de texto en unidades individuales o tokens para que las máquinas comprendan y manejen datos textuales de manera más eficiente

Types Of Tokenization

"Machine",
"learning",
"is", "fun", "."

Word-Based

"ma",
"chine",
"learn", "ing"

Subword-Based

"M", "a", "c",
"h", "i", "n",
"e", "l", "e",
"a", "r", "n",
"i", "n", "g"

Character-Based



LENGUAJES Y COMPILADORES

TOKENIZACION EN LENGUAJES FORMALES

Se refiere a la primera etapa de las fases de un compilador el cual comprende el análisis léxico donde los token están basados en las reglas del lenguaje

Tokens		
(id,"x")	(op,":=")	(id,"a")
(op,"+")	(id,"b")	(op,"*")
(id,"c")	(punct,";")	
(id,"y")	(op,":=")	(id,"3")
(op,"+")	(id,"b")	(op,"*")
(id,"c")	(punct,";")	



LENGUAJES Y COMPILADORES

TOKENIZACION EN EL NLP

En el NLP la tokenizacion se aborda de diferente manera de segun los requerimientos especificos de la tarea y sirve a multiples propositos como por ejemplo clasificar textos

Tokenization

Natural Language Processing

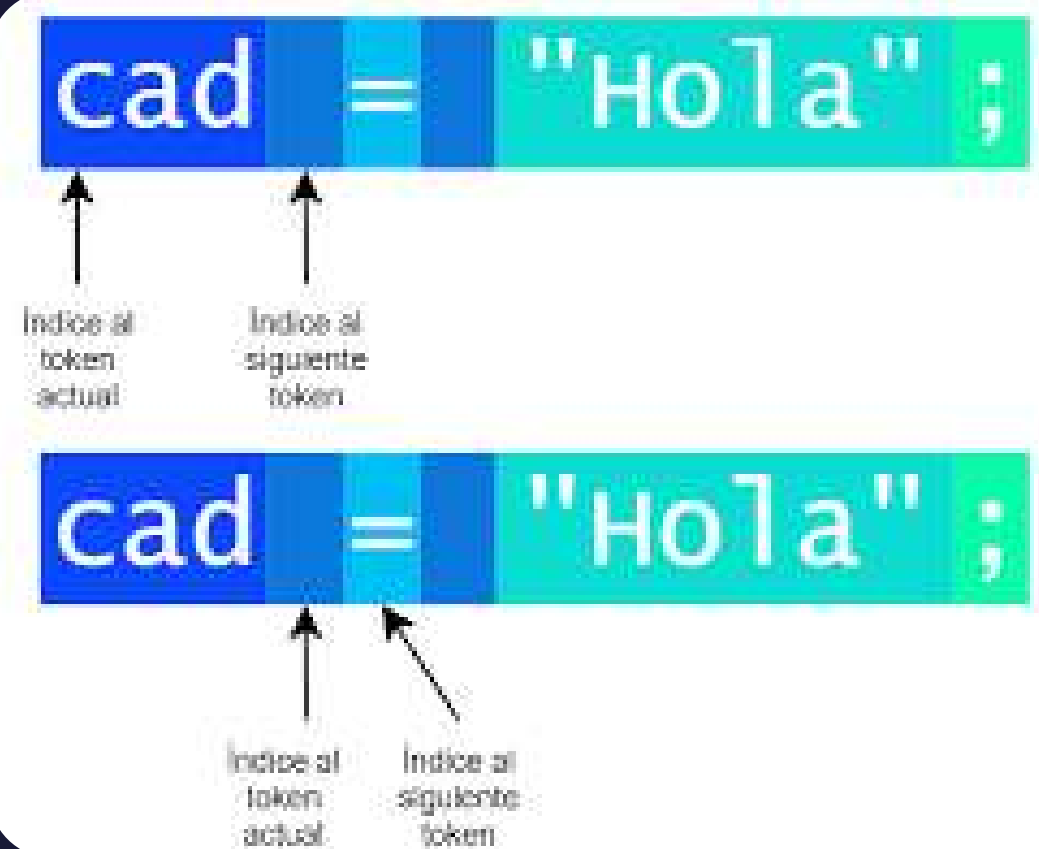


['Natural', 'Language', 'Processing']



LENGUAJES Y COMPILADORES

**¿CUÁL ES SU
IMPLICACIÓN CON LOS
COMPILADORES?**





LENGUAJES Y COMPILADORES

EL ANÁLISIS LÉXICO: EL "SCANNER"

El compilador utiliza un componente llamado Scanner. Su trabajo es actuar como un filtro de alta velocidad.





LENGUAJES Y COMPILADORES

IMPLICACIONES TÉCNICAS: LA CONEXIÓN CON EL PARSER Y LA TABLA DE SÍMBOLOS

El Parser no analiza letras, analiza la jerarquía de los tokens. Si el léxico le entrega una secuencia como ID, SIGNO_IGUAL, NUMERO, el Parser sabe que está ante una asignación.

DE \ PARA	<i>byte</i>	<i>short</i>	<i>char</i>	<i>int</i>	<i>long</i>	<i>float</i>	<i>double</i>
<i>byte</i>		Implicito	char	Implicito	Implicito	Implicito	Implicito
<i>short</i>	byte		char	Implicito	Implicito	Implicito	Implicito
<i>char</i>	byte	short		Implicito	Implicito	Implicito	Implicito
<i>int</i>	byte	short	char		Implicito	Implicito	Implicito
<i>long</i>	byte	short	char	int		Implicito	Implicito
<i>float</i>	byte	short	char	int	long		Implicito
<i>double</i>	byte	short	char	int	long	float	



LENGUAJES Y COMPILADORES

IMPORTANCIA Y EFICIENCIA

Más allá de la organización, hay una razón de rendimiento. Procesar texto es costoso. Al convertir strings en tokens el compilador acelera drásticamente su velocidad de procesamiento.





LENGUAJES Y COMPILADORES

NPL Y IA



El procesamiento del lenguaje natural o NPL es el campo de la inteligencia artificial que permite que las máquinas entiendan, procesen y respondan al lenguaje humano. Sin esta capacidad no existirían los asistentes virtuales, los chatbots, los traductores ni los modelos de texto que usamos hoy diario



LENGUAJES Y COMPILADORES

FUNCION DEL NPL EN LA IA



Este tiene como función tomar enormes volúmenes de texto, aprende significados, relaciones y patrones entre palabras y después aplica ese conocimiento para la interpretación de frases nuevas. Aplica métodos como la tokenización, embeddings , entre otras más.



LENGUAJES Y COMPILADORES

EL PROBLEMA DE OOV



Cuando una IA encuentra una palabra que no está en su base de datos original, el sistema colapsa o pierde el contexto.

Los modelos modernos resuelven esto no memorizando palabras completas, sino descomponiéndolas en fragmentos más pequeños.



LENGUAJES Y COMPILADORES

SOLUCION AL OOV

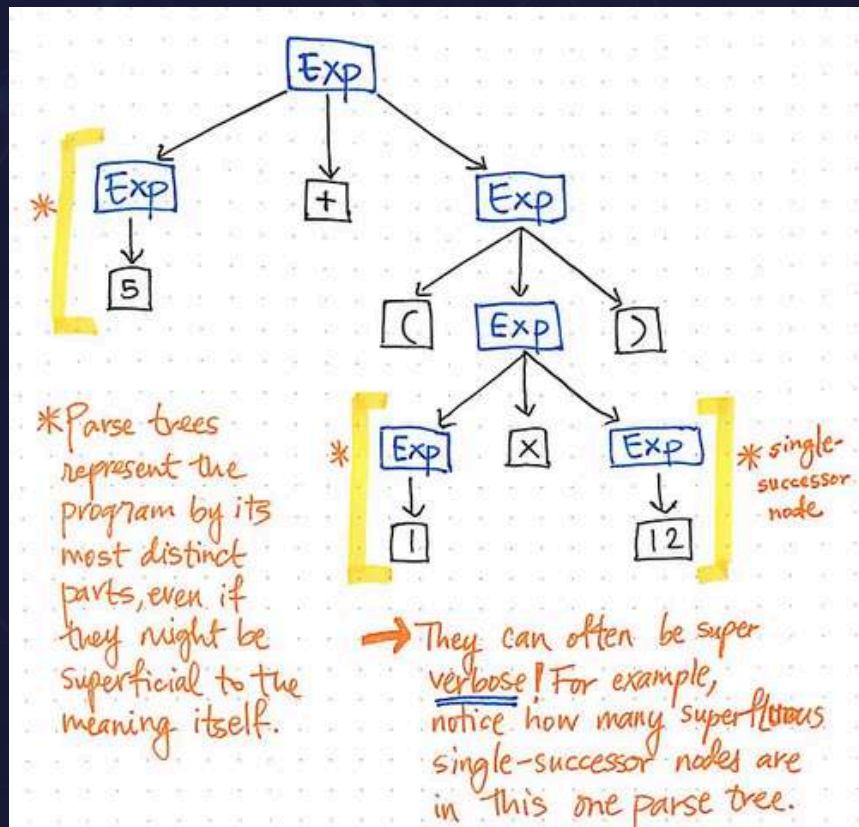


Para solucionar este problema la IA moderna utiliza la tokenización por subpalabras, esta consiste en dividir el texto en unidades mayores que un solo carácter pero menores que una palabra completa. Esto resulta útil al trabajar con palabras poco frecuentes.



LENGUAJES Y COMPILADORES

BLOQUES DE CONSTRUCCIÓN



En el corazón de ambos sistemas reside la definición de "unidad mínima".

- Compiladores: Utilizan lexemas rígidos definidos por expresiones regulares.
- NLP: Utiliza unidades fluidas que dependen del contexto semántico.



LENGUAJES Y COMPILADORES

MANEJO DE ERRORES DE SINTAXIS

Rigidez del Compilador

Ante un símbolo no válido o una gramática rota, el proceso se detiene. El sistema es binario: el código es válido o es basura.

Adaptabilidad en NLP

Usa probabilidades para intuir la intención. Ante una palabra desconocida (OOV), la descompone en sub-partes para no detener el flujo.





LENGUAJES Y COMPILADORES

SEPARACIÓN Y CONCATENACIÓN

¿Cómo decidimos dónde termina una idea y empieza otra?

Separación

- Los compiladores ignoran espacios/ruido.
- El NLP debe analizarlos

(ej. "San Francisco" vs "San" y "Francisco").

Concatenación

- En compiladores es aritmética
- En NLP es semántica, reconstruir matices y sentimientos.



LENGUAJES Y COMPILADORES

RESUMEN DE DIFERENCIAS CLAVE

Atributo	Compiladores (Leng. Formal)	NLP (Leng. Natural)
Unidad Base	LEXEMA Estático y finito.	TOKEN / VECTOR Dinámico y basado en contexto.
Tolerancia	CERO Error detiene la ejecución.	ALTA Busca la interpretación más probable.
Ruido	FILTRADO Elimina espacios y comentarios.	ANALIZADO Los espacios pueden ser tokens con significado.